



# Curso de Desarrollo en React

## Contraseña de equipos y WIFI

- Usuario y contraseña de los equipos:
  - pmpcurso1
  - c12345678\*
- WIFI SSID y contraseña WIFI
  - TP-Link\_3376
  - TP-Tracasa2024\*\*

# Índice

1. Contexto
  2. Entorno de desarrollo
  3. Metodología
  4. Desarrollo de componentes
  5. Práctica I
- 
6. Estructuración código
  7. Pruebas
  8. Práctica II
  9. Implementación de componentes
  10. Cascading Style Sheets (CSS)
  11. CI/CD
  12. Práctica III

## Contexto

- Cambio de tecnología
  - Inicio en la 5.7
  - Pantallas nuevas y actuales
- Antes de comenzar...
  - **Formación previa en React** (vídeos formativos).
  - **Formación previa de CSS** (tareas interactivas).
  - Tener instalado **Visual Studio Code, Node y Git**.
  - Clonar/Descargar el **repositorio**:  
<https://github.com/julenTracasa/Formacion-React>

## GIT: Gestión de Ramas

- Rama Main
  - Rama del equipo (Obligatorio)
    - Rama por "feat" (Recomendada)

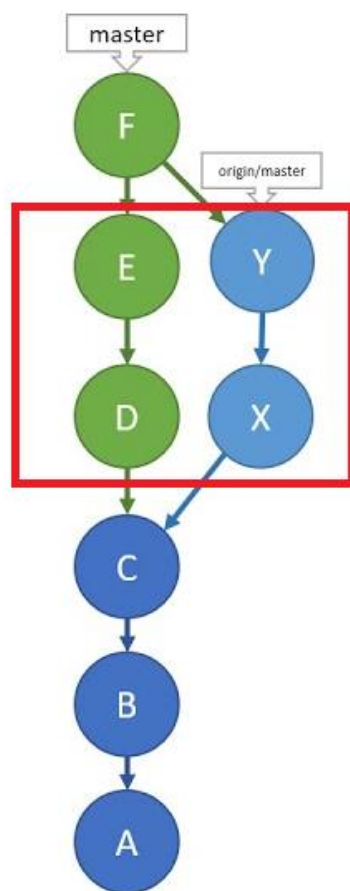
- **No se puede hacer commit directamente a Main**

## GIT: Commit / push

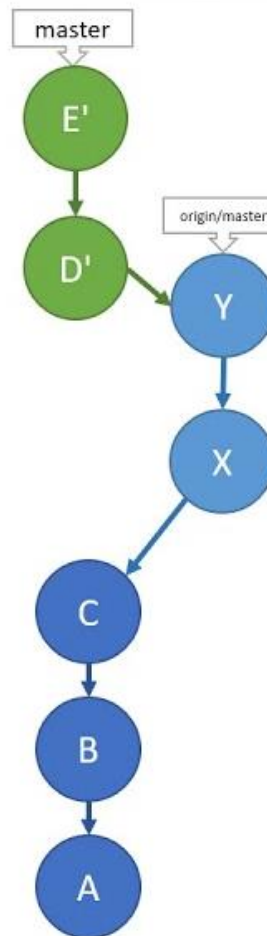
- Commit
  - Guarda los cambios que se hayan realizado en local
  - Para asociar Tasks hay que añadir al comentario del commit `"#{numeroDeLaTask}"`
- Push
  - Tener instalado **Visual Studio Code**, **Node** y **Git**.
  - Clonar/Descargar el **repositorio**:  
<https://github.com/julenTracasa/Formacion-React>

## GIT: Pull (Rebase)

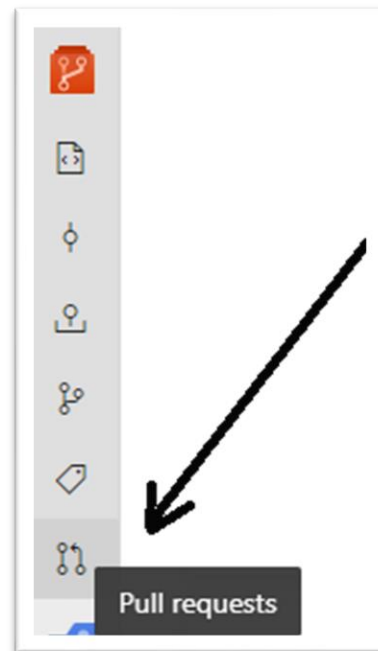
```
git pull origin master
```



```
git pull --rebase origin master
```



# GIT: Pull Request



Solucion de descarga de grabaciones #489431

Completed 11492 Arkaitz Calvo Saldias (ext) green-day into main

Overview Files Updates Commits

Arkaitz Calvo Saldias (ext) completed this pull request 1h ago [Cherry-pick](#) [Revert](#)

Solicitud de incorporación de cambios fusionados 1492: Solucion de des...  
61889533 Arkaitz Calvo Saldias (ext) Today at 9:09

[Show details](#)

1 reviewer approved

No merge conflicts  
Last checked 1h ago

Description

Solucion de descarga de grabaciones 489431 Desarrollo • New

Delete source branch

Reviewers

Required

No required reviewers

Optional

Arkaitz Calvo Saldias (ext)  
Approved

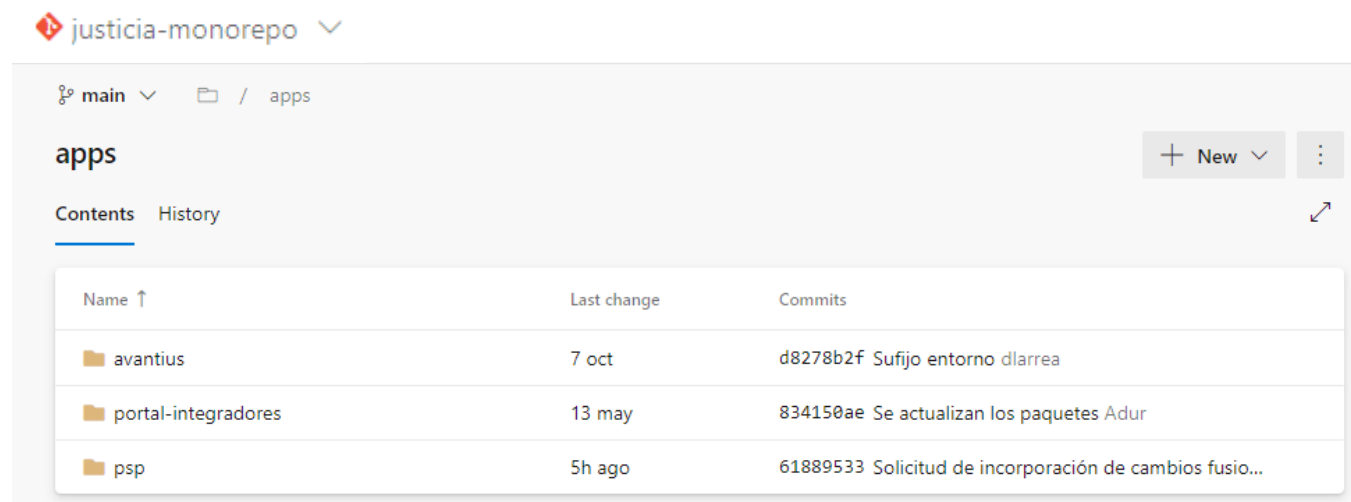
Tags

No tags

Work items

Task 489431: Desarrollo  
Updated 43m ago, New

# GIT: Monorepositorio



justicia-monorepo

main / apps

apps

+ New

Contents History

| Name ↑              | Last change | Commits                                                 |
|---------------------|-------------|---------------------------------------------------------|
| avantius            | 7 oct       | d8278b2f Sufijo entorno dlarrea                         |
| portal-integradores | 13 may      | 834150ae Se actualizan los paquetes Adur                |
| psp                 | 5h ago      | 61889533 Solicitud de incorporación de cambios fusio... |

# Diseño atómico

Esencia **diseño atómico**: Las interfaces están formadas por componentes más pequeños. Podemos **dividir interfaces** completas en **bloques de construcción** fundamentales y trabajar desde allí.

Nos obliga a coger cada diseño y **diseccionarlo** para poder separar cada átomo.



Átomos



Moléculas



Organismos



Plantillas



Páginas

# Diseño atómico: Organismos

## INTERVINEINTES

|                                                                                     |                                                          |                                              |
|-------------------------------------------------------------------------------------|----------------------------------------------------------|----------------------------------------------|
|    | <b>Alejandro José Martínez Faci</b><br>Demandado         | ...                                          |
| Defensor<br>Procurador                                                              |                                                          | Usuario Abogado Avantius<br>Justicia pruebas |
|    | <b>Javier Ganuza Alonso</b><br>Demandante                | ...                                          |
| Defensor<br>Procurador                                                              |                                                          | Añadir<br>Añadir                             |
|    | <b>Carla Jiménez Almagro</b><br>Capitán de buque         | ...                                          |
| Defensor<br>Procurador                                                              |                                                          | Añadir<br>Añadir                             |
|  | <b>Orkatz Aldave Alustiza</b><br>Administrador Concursal | ...                                          |
| Defensor<br>Procurador                                                              |                                                          | Añadir<br>Añadir                             |

## Diseño atómico: Moléculas



**Alejandro José Martínez Faci**

Demandado



Defensor  
Procurador

Usuario Abogado Avantius  
Justicia pruebas

# Diseño atómico: Atomos

## GRANDE

48px, Regular stroke (2px)



## Heading 1

## HEADING 2

## Heading 3

Subtitle 1

## Body 1 (SemiBold)

Body 1 (Regular)

Body 2 (SemiBold)

Body 2 (Regular)

## TERCIARIO

Pequeño

Enlace

Default

Default

Default

Default

# Historia de usuario de ejemplo

## Campo de procedimiento

Procedimiento

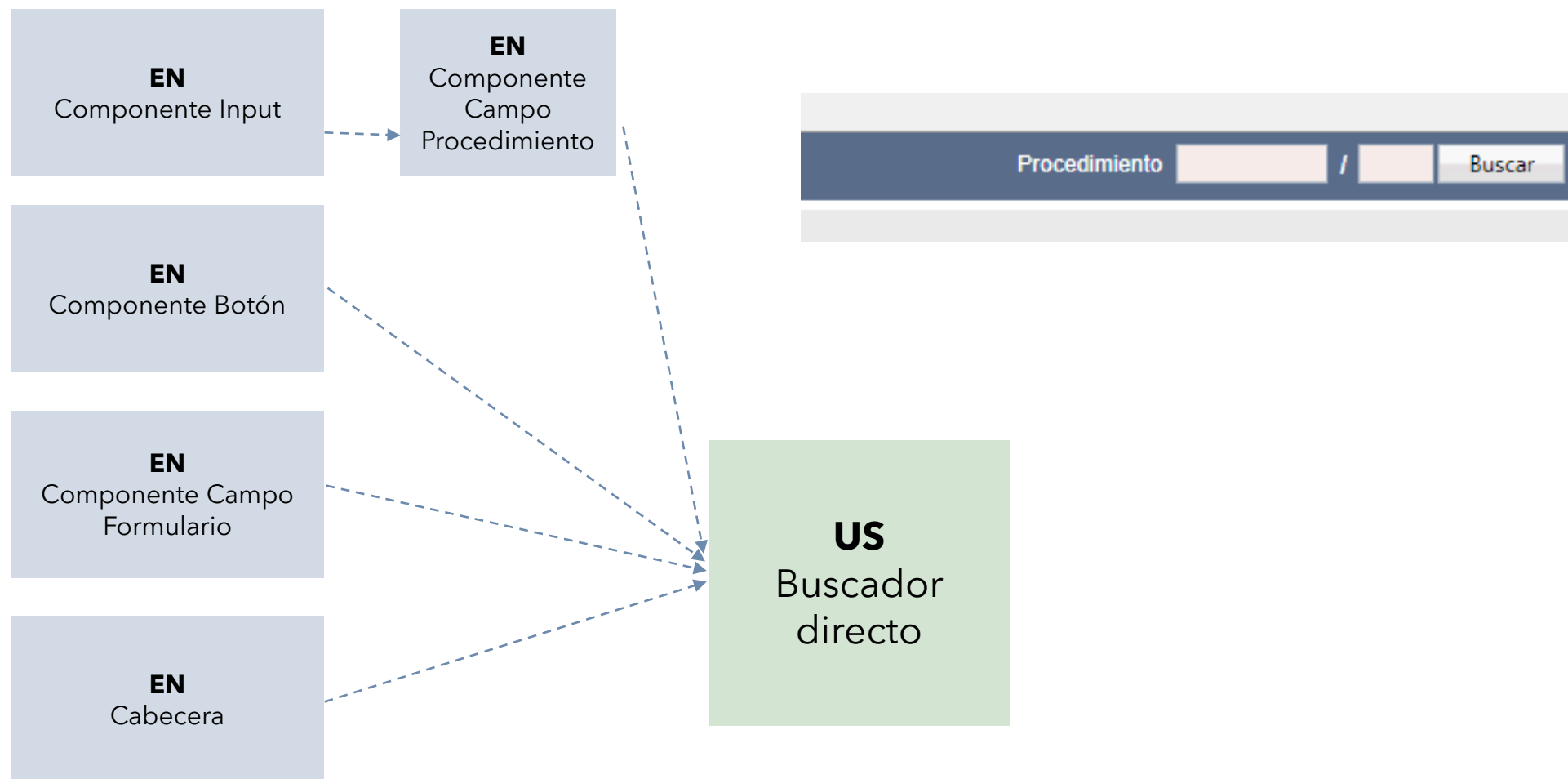
/

Buscar

| Órgano destinatario |                                                                                     |                                                                                     |
|---------------------|-------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------|
|                     |  |  |
| J.1ª Instancia nº 1 |  |  |
| J.1ª Instancia nº 1 |  |  |

# Historia de usuario de ejemplo

## Campo de procedimiento



# Desarrollo de un componente:

- 1. Definición de requisitos**
- 2. Diseño**
3. Desarrollo
4. Pruebas
5. Implementación

# Desarrollo de un componente: Definición requisitos

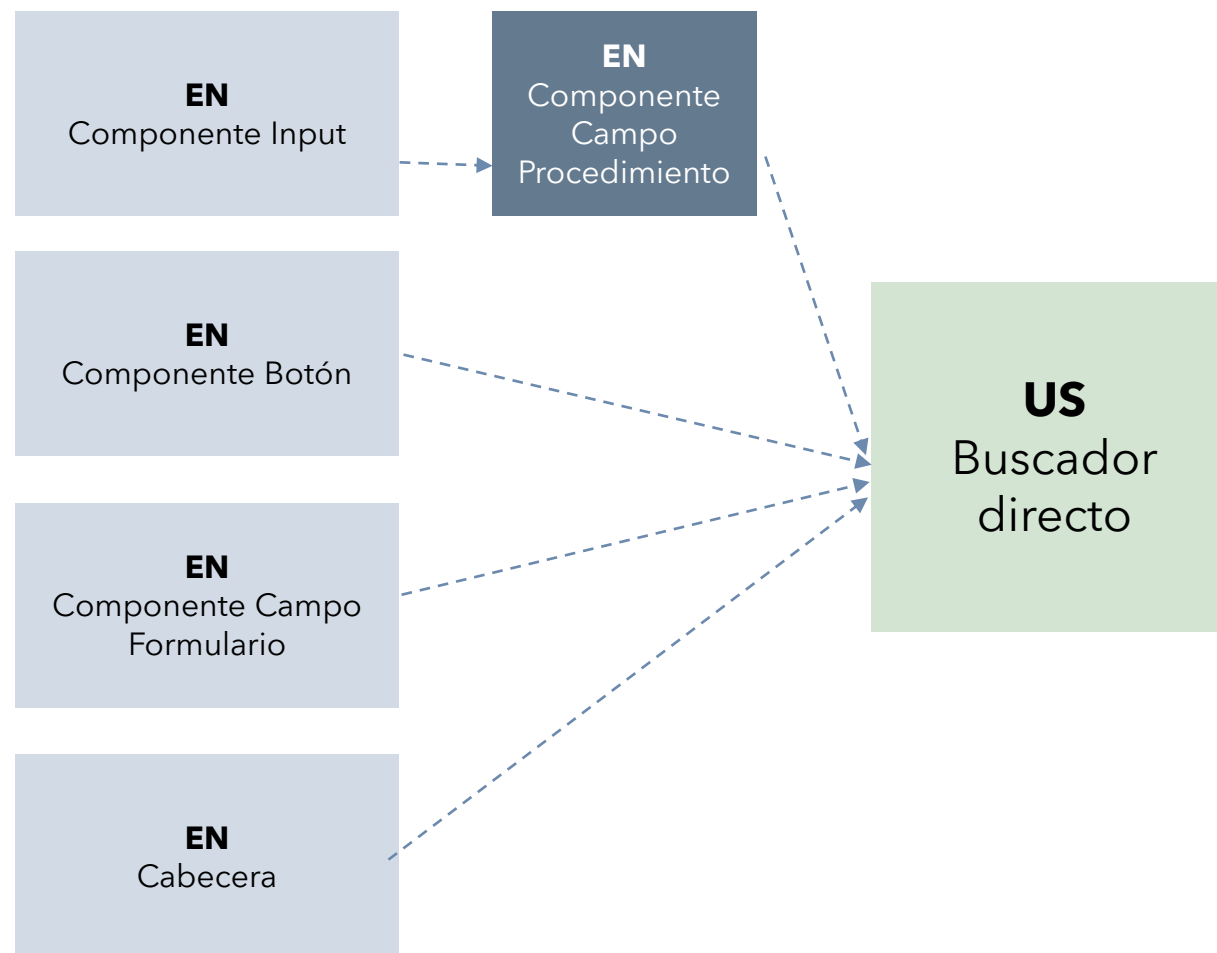
- **Accesibilidad**
  - Navegación con teclado.
  - Orden de tabulación lógico.
  - Lectura del componente por el Narrador:
    1. ¿Qué información es relevante para el usuario?
- **Requisitos** funcionales:
  - Aplicables a **nuestro caso de uso**.
  - **Globales**.

## Desarrollo de un componente: Diseño

1. Revisar el **Design System** y el **Storybook**.
  - ¿Se pueden reutilizar componentes del Storybook?
  - ¿Se pueden reutilizar diseños del DS?
  - ¿Dudas? Consulta a Rolling Styles.
2. Realizar un diseño en Figma siguiendo el **Design System**.
  - Consultar a Rolling Styles todas las **dudas** respecto al **UX/UI**: Usabilidad, espaciados, colores.
3. Asegurarse de que Rolling Styles da **visto bueno al diseño** (Canal de Teams).
  - Enviar un caso de uso.

# Historia de usuario de ejemplo

## Campo de procedimiento



# Desarrollo de un componente: Definición requisitos

## Campo de procedimiento

- **Accesibilidad**

 /  

- **Requisitos** funcionales:

- Debe contener tres campos de texto: Número, año y pieza.
  - Validación del año.
    - Solamente se admiten valores numéricos.
    - No se admiten años inferiores a 1900.
    - No se admiten años superiores al actual.
  - Validación del número de procedimiento
    - Solamente se admiten valores numéricos.
    - No puede contener más de 7 dígitos.
    - En caso de tener menos, se autocompletará con "0" a la izquierda hasta rellenar los 7 dígitos.
  - Validación del número de la pieza
    - Solamente se admiten valores numéricos.
    - No puede contener más de 2 dígitos.

# Desarrollo de un componente: Diseño

## Campo de procedimiento

1. Revisar el **Design System** y el **Storybook**.

### INPUT

A collection of input field variants from a design system, displayed in a vertical list. The variants are: Default, Icono a la derecha (with a calendar icon), Icono a la izquierda (with a magnifying glass icon), Placeholder, Read-Only, Disabled, Focus (with a blue border and a vertical line), Typing (with a blue border and a vertical line), Error (with a red border), Focus (with a red border and a vertical line), and Typing (with a red border and a vertical line). The first four variants are grouped together in a dashed purple box.

# Desarrollo de un componente: Diseño Campo de procedimiento

## 2. Realizar un diseño en Figma siguiendo el **Design System**

### CAMPO PROCEDIMIENTO

The 'CAMPO PROCEDIMIENTO' design shows a grid of input fields with different states and combinations of values. The fields are arranged in a grid with 4 columns and 8 rows. The first three columns represent the 'Número', 'Año', and '00' fields, and the last column represents the 'Número', 'Año', and '00' fields. The states shown are: Default, Read-Only, Disabled, Focus, Typing, and Error. The values shown are: 0000001, 2023, 01, 1899, and 00. The design is enclosed in a dashed purple box.

|         |      |    |         |      |    |
|---------|------|----|---------|------|----|
| Default | 2023 | 01 | Número  | Año  | 00 |
| 0000001 | 2023 | 01 | 0000001 | 2023 | 01 |
| 0000001 | 2023 | 01 | 0000001 | 1899 | 01 |
| 0000001 | 1899 | 01 | Default | 2023 |    |
| Número  | Año  |    | 0000001 | 2023 |    |
| 0000001 | 2023 |    | 0000001 | 2023 |    |
| 0000001 | 1899 |    | 0000001 | 1899 |    |

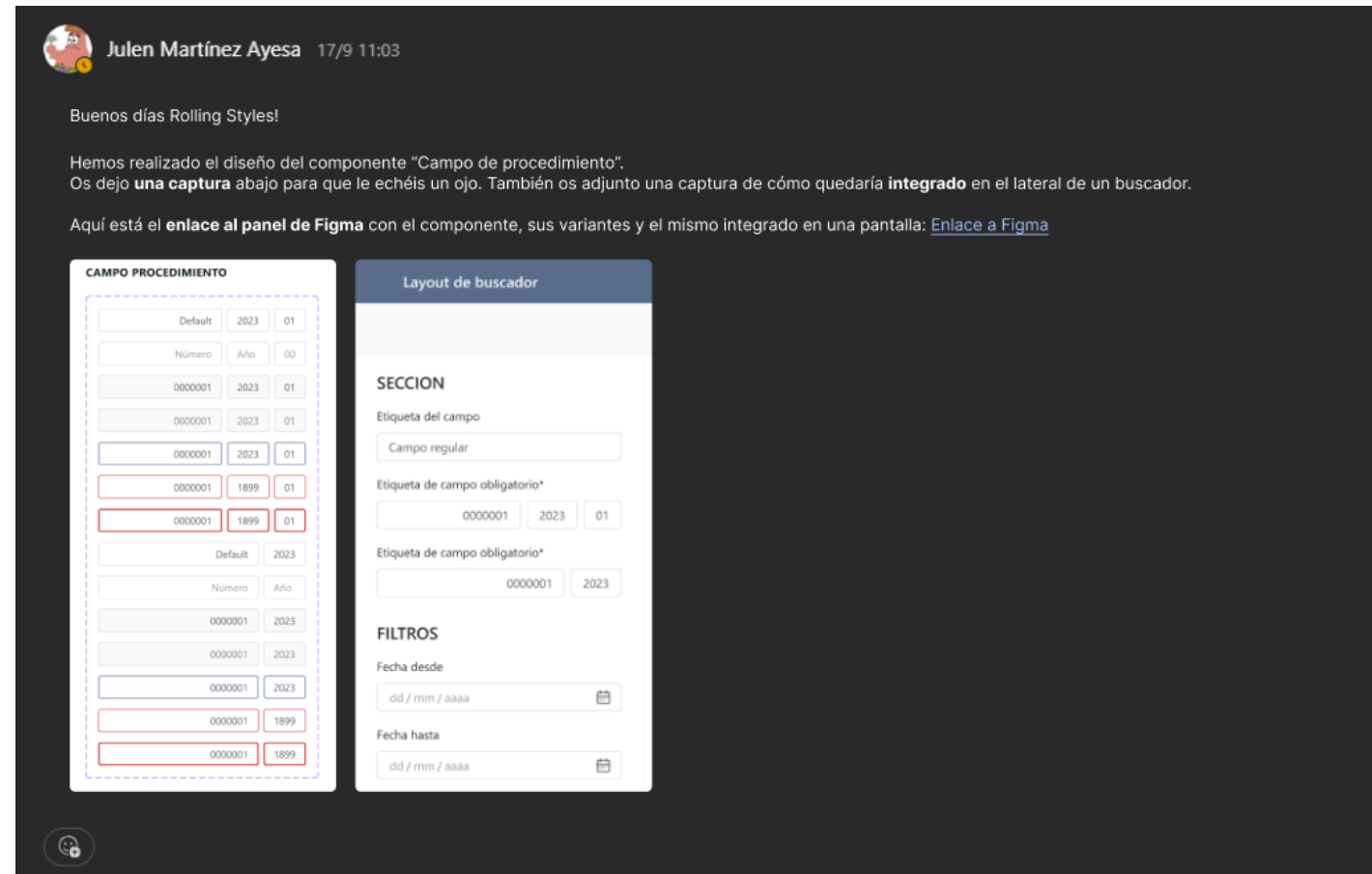
### INPUT

The 'INPUT' design shows a vertical list of input fields with different states and combinations of values. The fields are arranged in a vertical list. The states shown are: Default, Icono a la derecha, Icono a la izquierda, Placeholder, Read-Only, Disabled, Focus, Typing, Error, Focus, and Typing. The values shown are: Default, 0000001, 2023, 01, 1899, and 00. The design is enclosed in a dashed purple box.

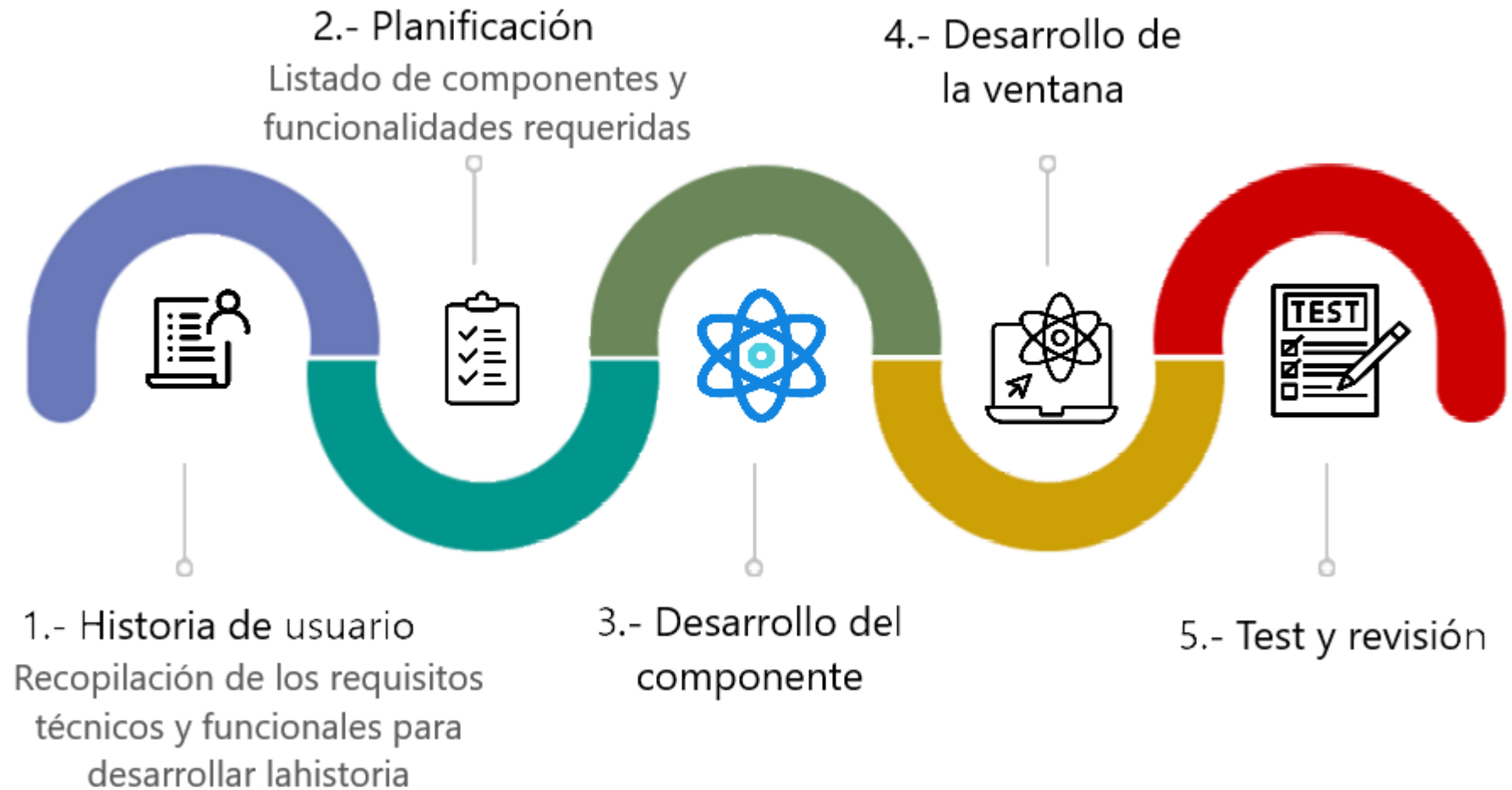
|                      |
|----------------------|
| Default              |
| Icono a la derecha   |
| Icono a la izquierda |
| Placeholder          |
| Read-Only            |
| Disabled             |
| Focus                |
| Typing               |
| Error                |
| Focus                |
| Typing               |

# Desarrollo de un componente: Diseño Campo de procedimiento

3. Asegurarse de que Rolling Styles da **visto bueno al diseño** (Canal de Teams).



## Ciclo de desarrollo



## Ciclo de desarrollo

### 1. ¿Qué componentes necesita la historia de usuario?

#### 1. Buscador

##### 1. Formulario

##### 1. Campo Procedimiento

##### 2. Campo Etiquetas

##### 3. Selector Simple

....

#### 2. Listado Paginado

▸  GdN\_PSP - MULTI-DISPOSITIVO – FASE 1. Índice electrónico. ENABLER - Componentes buscador de expedientes 2/3

▸  GdN\_PSP - MULTI-DISPOSITIVO – FASE 1. Índice electrónico. Buscar expedientes desde el buscador 1/3

## Desarrollo de un componente

- Desarrollar a nivel de:
  - Ventana (si es un componente único)
  - Aplicación (si es exclusivo de esta)
  - Librería de componentes (abierto para emplear en otros casos)
- Importante:
  - Emplear los hooks necesarios
  - Buenas prácticas/patrones (Compound, HOC, ...)

# Ejercicio 1: Creación del Campo de Procedimiento (45 min)

## Requisitos a nivel de testing (5 min)

## Requisitos a nivel de testing (5 min)

- **Accesibilidad**
- **Requisitos** funcionales:
  - Debe contener tres campos de texto: Número, año y pieza.
    - Validación del año.
      - Solamente se admiten valores numéricos.
      - No se admiten años inferiores a 1900.
      - No se admiten años superiores al actual.
    - Validación del número de procedimiento
      - Solamente se admiten valores numéricos.
      - No puede contener más de 7 dígitos.
      - En caso de tener menos, se autocompletará con "0" a la izquierda hasta rellenar los 7 dígitos.
    - Validación del número de la pieza
      - Solamente se admiten valores numéricos.
      - No puede contener más de 2 dígitos.

# Cómo hacer un test (15-20 min)

## Ejercicio 2: Solucionar los test del componente Button (15 min)

## Estructura de carpetas

- public (imágenes, literales, pdfjs, ...)
- src
  - pages
  - shared (componentes, configs, hooks, entities, http, zustand, providers, helpers)
  - styles (index.css, reset.css)
  - test (renders para test, utilidades para tests, setup...)
  - Workers
  - App.tsx
  - app.router.tsx
  - main.tsx

## Estructura de carpetas

### **Capa de Presentación (UI Layer):**

- pages/
- shared/components/
- styles/

### **Capa de Negocio (Business Logic Layer):**

- shared/hooks/
- shared/entities/
- shared/providers/

### **Capa de Datos (Data Layer):**

- shared/http/
- shared/zustand/

### **Capa de Utilidades (Utility Layer):**

- shared/configs/
- shared/helpers/

### **Capa de Pruebas (Testing Layer):**

- test/
- Cada componente

# Desarrollo de una ventana

## Archivos

- **.tsx**
  - inyecta dependencias a .page.tsx
- **.page.tsx**
  - importa .service.tsx, .store.tsx y .module.css
  - Contiene la lógica de la ventana
- **.service.tsx**
  - Uso de repositorio
  - Mapear DTOs
- **.store.tsx (Opcional)**
  - Estados compartidos entre páginas/componentes
- **.page.test.tsx (Opcional)**
- **.module.css (Opcional)**
- **shared/http/http.ts (Opcional)**
  - Llamadas http y definición del repositorio

# Desarrollo de una ventana

## Literales

- i18n

```
apps > psp > public > locales > {} es.json > {} commons
1  {
2    "errores": {
3      "intenteloDeNuevo": "Ha ocurrido un
4    },
5    "commons": {
6      "cargando": "Cargando",
7      "reintentar": "Reintentar",
8      "si": "Sí",
9      "no": "No",
10     "fecha": "Fecha",
```

```
const { t } = useTranslation();
```

```
<Section
  id="filtros"
  title={t('buscador-expedientes.filtro')}
  mostrarTitulo={false}
>
```

# Desarrollo de una ventana

## Contextos

- Zustand
  - Objetos (sin métodos, solo valores)
  - Cachear
- Context
  - Objetos (con métodos)
  - Necesita de más boilerplate (Providers)
  - Renderizado al cambiar objetos del Context

# Desarrollo de una ventana

## Routing

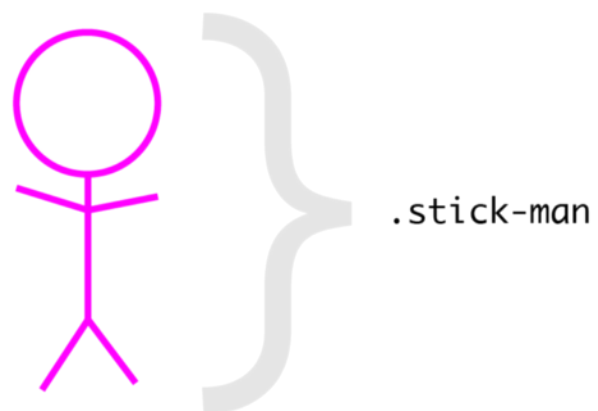
- `app.router.tsx`
  - Lazy
  - HashRouting
  - Layout
- `src/shared/config/routing/index.ts`

# Cascading Style Sheets (CSS): Nomenclatura

- **B**loques
- **E**lementos
- **M**odificadores

# Cascading Style Sheets (CSS): Nomenclatura

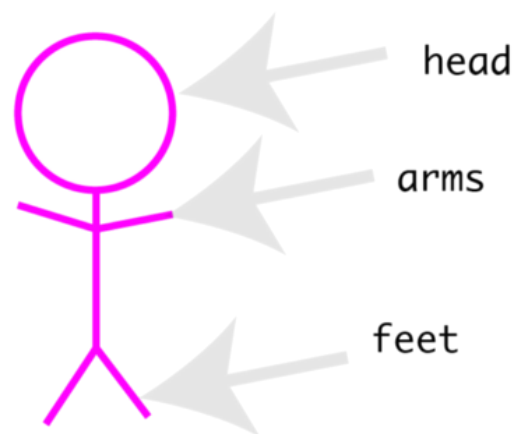
**Bloque:** Entidad independiente con significado por sí misma.



```
.stick-man {  
  ...  
}
```

# Cascading Style Sheets (CSS): Nomenclatura

**Elemento:** Parte de un bloque. No tiene significado por sí sola.



```
.stick-man__head {  
  ...  
}  
  
.stick-man__arms {  
  ...  
}  
  
.stick-man__feet {  
  ...  
}
```

# Cascading Style Sheets (CSS): Nomenclatura

**M**odificcador: Parte de un bloque. No tiene significado por sí sola.



blue stick man



red stick man

```
.stick-man--blue {  
  ...  
}  
  
.stick-man--red {  
  ...  
}
```

# Cascading Style Sheets (CSS): Selectores

- **Básicos**

- **Universal** `* { ... }`
- **Elemento** `div { ... }`
- **Clase** `.clase { ... }`
- **Identificador** `#identificador { ... }`

- **Combinado**

- **Descendiente** `div a { ... }`
- **Hijo directo**
- **Hermano**
- **Hijo adyacente**
- **OR** `div, a { ... }`

# Cascading Style Sheets (CSS): Selectores

- Atributos

- **Estados**

- **Hover**

```
input:hover { ... }
```

- **Focus**

```
input:focus { ... }
```

- **Required**

```
input:required { ... }
```

- **Checked**

```
input:checked { ... }
```

- **Disabled**

```
input:disabled { ... }
```

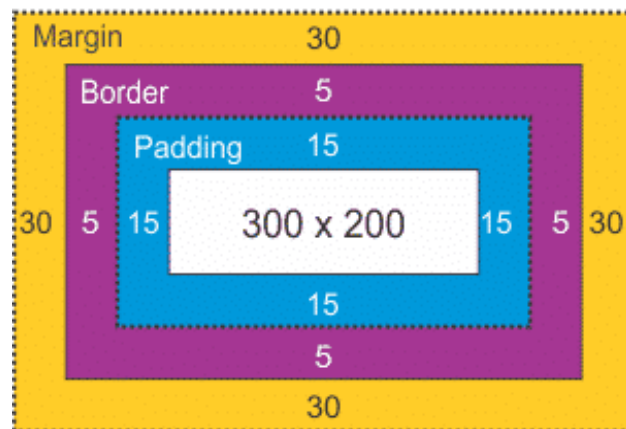
- Pseudo elementos

- Before

- After

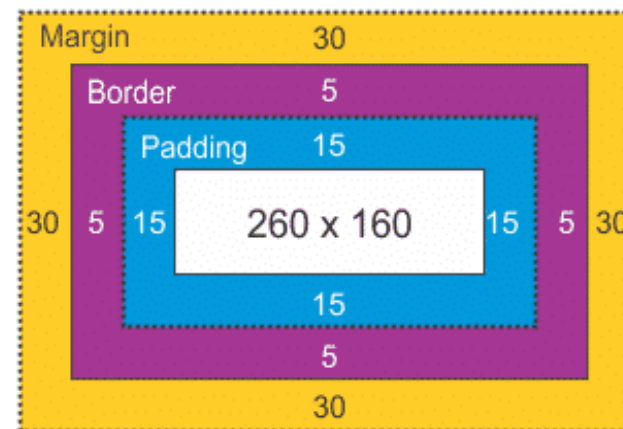
# Cascading Style Sheets (CSS): Box model

Box Model is content-box



```
div{  
  width: 300px;  
  height: 200px;  
  padding: 15px;  
  border: 5px solid grey;  
  margin: 30px;  
  -moz-box-sizing: content-box;  
  -webkit-box-sizing: content-box;  
  box-sizing: content-box;  
}
```

Box Model is border-box



```
div{  
  width: 300px;  
  height: 200px;  
  padding: 15px;  
  border: 5px solid grey;  
  margin: 30px;  
  -moz-box-sizing: border-box;  
  -webkit-box-sizing: border-box;  
  box-sizing: border-box;  
}
```

# Cascading Style Sheets (CSS): Flexbox

`display: flex`

1. `flex-direction`
2. `justify-content`
3. `align-items`

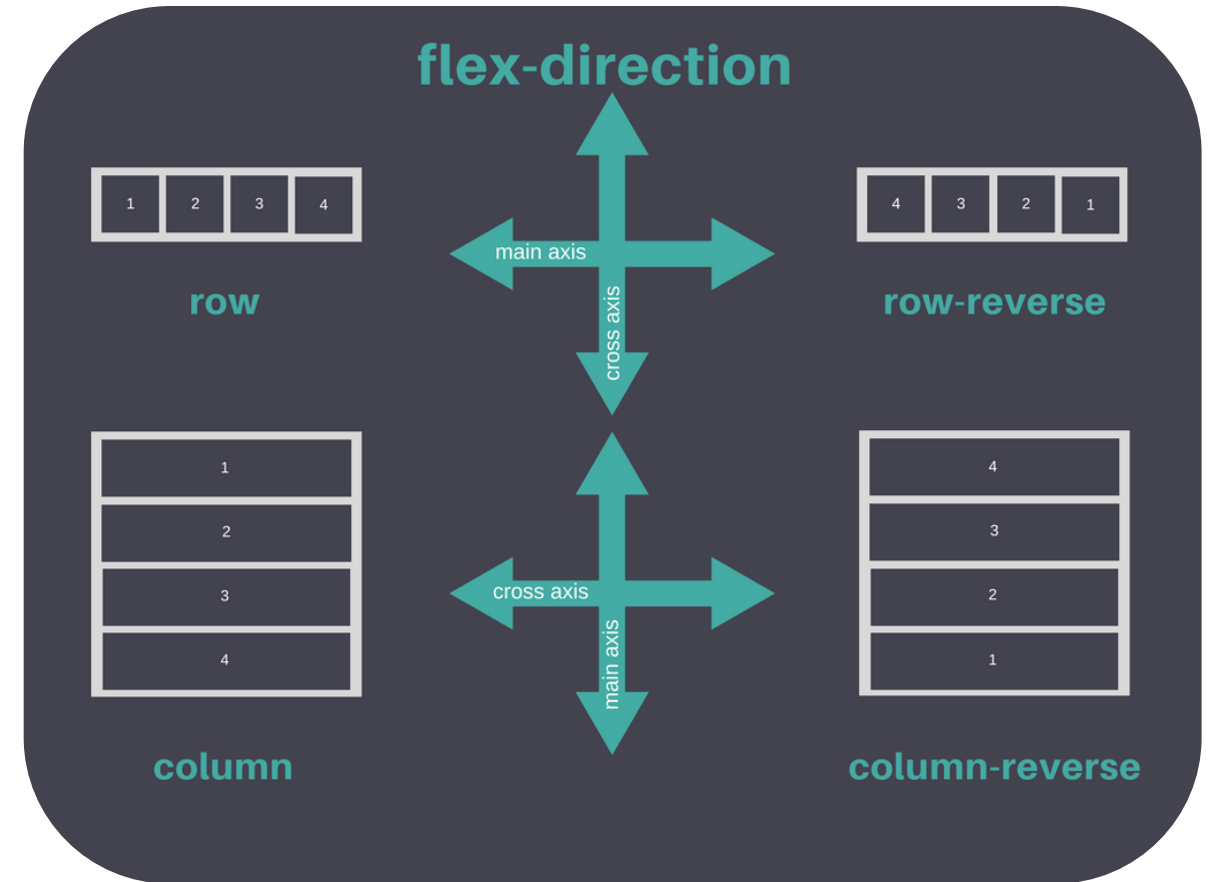
# Cascading Style Sheets (CSS): Flexbox

```
contenedor {  
  display: flex  
  flex-direction: column  
}
```

Dos ejes:

- Eje Principal
- Eje Transversal

Se especifica **dirección del Eje Principal**. El valor predeterminado es "row".



# Cascading Style Sheets (CSS): Flexbox

La justificación actúa sobre el **Eje Principal**.  
El valor predeterminado es "flex-start".

```
contenedor {  
  display: flex  
  justify-content: flex-start  
}
```

flex-start



flex-end



center



space-between



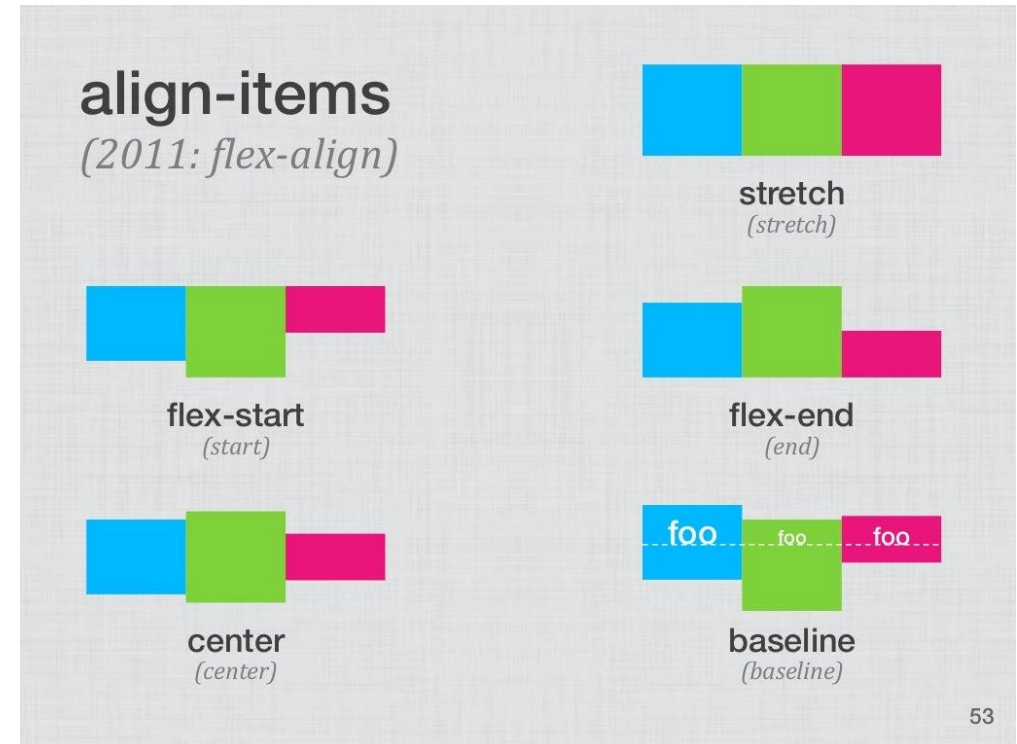
space-around



# Cascading Style Sheets (CSS): Flexbox

El alineamiento actúa sobre el **Eje Transversal**. El valor predeterminado es "flex-start".

```
contenedor {  
  display: flex  
  align-items: flex-start  
}
```



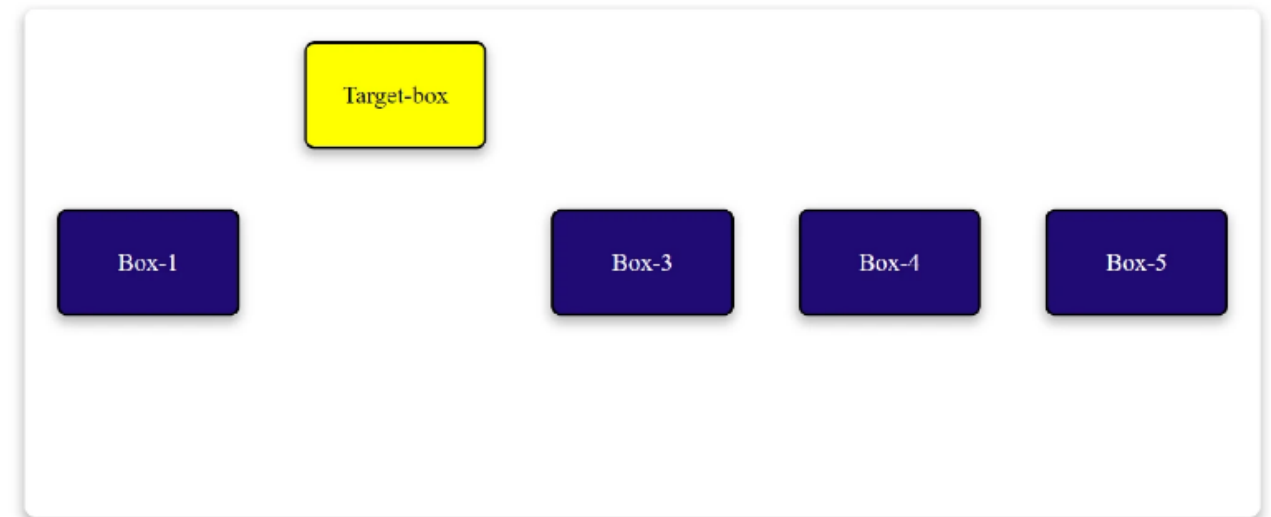
# Cascading Style Sheets (CSS): Flexbox

¿Y qué pasa si quieres que un elemento **rompa la norma**?

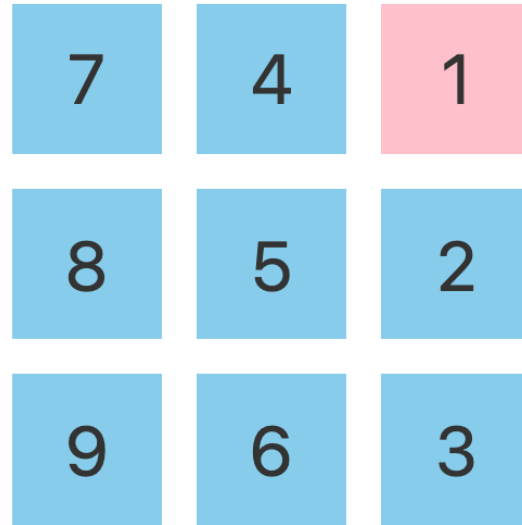
```
elemento {  
  justify-self: end  
}
```



```
elemento {  
  align-self: start  
}
```



# Cascading Style Sheets (CSS): Grid



display: grid;

```
1 .grid {  
2   width: 240px;  
3   height: 240px;  
4   display: grid;  
5   grid-template-columns: repeat(3, 1fr);  
6   grid-template-rows: repeat(3, 1fr);  
7   gap: 16px;  
8   grid-auto-flow: column;  
9   direction: rtl;  
10 }  
11
```

# Cascading Style Sheets (CSS): Buenas prácticas

- **Consistencia**
- Evitar **estilos en línea**
- **Reutilización** de estilos
- Evitar el uso de **!important**
- **CSS Modules** en React
- Los componentes deben ser **responsive**
- Minimizar lo máximo el uso de **comentarios**

[Artículo de la Wiki: Cascading Style Sheets \(CSS\)](#)

## Desarrollo de una ventana: CSS Modules

### buscadorExpedientes.module.css

```
.page {  
  flex: 1;  
  position: relative;  
  display: flex;  
  flex-direction: column;  
}
```

```
llamas@pmpwn538 C:\...\justicia-monorepo > green-day npm run generate-css-types
```

```
import styles from './buscadorExpedientes.module.css';  
  
<main className={styles.page}>
```

# Desarrollo de una ventana:

## Llamadas a back-end

```
export const useBuscadorExpedientesHttp = (): BuscadorExpedientesRepository => {  
  return {  
    obtenerMaestros: (request: BuscadorExpedientesObtenerMaestrosRequest) => {  
      return fetchApiRequestWsPSP<  
        BuscadorExpedientesObtenerMaestrosRequest,  
        BuscadorExpedientesObtenerMaestrosResponse  
      >({  
        httpMethod: 'POST',  
        path: '/BuscadorExpedientes/ObtenerMaestros',  
        data: request,  
      }));  
    },  
    buscar: (request: BuscadorExpedientesBusquedaRequest) => {  
      return fetchApiRequestWsPSP<  
        BuscadorExpedientesBusquedaRequest,  
        BuscadorExpedientesBusquedaResponse  
      >({  
        httpMethod: 'POST',  
        path: '/BuscadorExpedientes/BuscarExpedientes',  
        data: request,  
      }));  
    },  
  },  
};
```

## Ejercicio 3: Creación de una página e implementación del componente (60 min)

# Desarrollo de una ventana: Mockeo llamadas a back-end

# Desarrollo contra publicado

## PSPProxy

# Pipelines

## Build + Release

¿Dudas?

# Creación de una aplicación nueva